🎲 TiDPI 04 — Dane deterministyczne, powtarzalność i złożoność

📋 Tematy do dziennika

```
[TiDPI] Dane deterministyczne i pseudolosowe – generator, ziarno, odcisk danych oraz odtwarzanie przypadku testowego
[TiDPI] Powtarzalność eksperymentu – kontrolowanie danych wejściowych, konfiguracji, procedury i wersji badanego programu
[TiDPI] Empiryczne badanie złożoności sortowania – liczba porównań i zamian, tempo wzrostu oraz wpływ układu danych
```

🎯 Cel bloku

Po tym bloku będziecie potrafili przygotować deterministyczne dane testowe, odtworzyć pseudolosowy przypadek za pomocą zapisanego ziarna (seed), odróżnić powtarzalność danych od stabilności czasu oraz wykorzystać liczbę operacji do zbadania, jak sortowanie bąbelkowe skaluje się dla danych posortowanych, odwróconych i pseudolosowych.

Rezultatem będzie jedno anonimowe sprawozdanie w Markdownie zawierające metryczkę eksperymentu, odciski danych, surowe wyniki, kontrolę poprawności, analizę tempa wzrostu i ostrożne wnioski.

Nasze główne pytanie brzmi:

> Jeśli uruchomimy ten sam program ponownie, skąd będziemy wiedzieli, że naprawdę powtórzyliśmy ten sam eksperyment?

Sam przycisk „Uruchom ponownie” nie wystarcza. Program może dostać inne dane, inną konfigurację, inną wersję kodu albo inne warunki wykonania. Powtórzenie eksperymentu zaczyna się więc od zapisania jego „przepisu”.

🧭 Determinizm — ta sama przyczyna, ten sam wynik

Deterministyczna operacja dla takich samych danych wejściowych i takiego samego stanu początkowego daje ten sam wynik. Przykładowo funkcja sortująca tę samą tablicę liczb powinna za każdym razem zwrócić taki sam uporządkowany ciąg.

Nie oznacza to, że każda właściwość uruchomienia będzie identyczna. Ten sam algorytm może wykonać te same porównania i zwrócić te same dane, ale potrzebować nieco innego czasu, ponieważ system operacyjny obsłużył w tle inne zadanie.

Rozdzielcie więc trzy rzeczy:

Pytanie	Co kontrolujemy?	Przykładowy dowód
Czy wejście było takie samo?	rozmiar, sposób generowania, seed i odcisk danych	identyczny `fingerprint`
Czy program zrobił to samo?	wersja kodu, algorytm, konfiguracja i wynik	te same liczniki operacji oraz `correct=true`
Czy trwało to dokładnie tyle samo?	wpływ środowiska i zakłóceń	zwykle nie; zapisujemy wszystkie czasy

Pseudolosowość i ziarno generatora

Programy często korzystają z **generatora liczb pseudolosowych** — PRNG (*Pseudo-Random Number Generator*). Jest to deterministyczny algorytm tworzący ciąg liczb wyglądający na losowy. Jego stan początkowy ustala **ziarno**, po angielsku *seed*.

Ten sam algorytm generatora uruchomiony z tym samym seedem tworzy ten sam ciąg swoich surowych wyników. Zmiana seeda zwykle prowadzi do innego ciągu. Dzięki temu można:

- odtworzyć przypadek, który ujawnił błąd;
- przekazać innemu testerowi dokładny przepis na dane;
- porównać dwie wersje programu na identycznym wejściu;
- zachować wiele kontrolowanych zestawów bez wpisywania tysięcy liczb do raportu.

W programie użyjemy generatora `std::mt19937`:

```
std::mt19937 generator(seed);
const std::uint32_t raw_value = generator();
```

`mt19937` to nazwa konkretnego algorytmu, a nie ogólne polecenie „daj coś losowego”. Sam zapis seeda bez nazwy generatora nie jest więc pełnym opisem danych.

Czy komputer potrafi uzyskać prawdziwą losowość?

Zdanie „komputery nie potrafią generować prawdziwej losowości” jest zbyt mocne. System może korzystać ze źródeł entropii pochodzących ze sprzętu lub zjawisk fizycznych. W tej lekcji celowo wybieramy jednak generator pseudolosowy, ponieważ chcemy odtwarzać dane, a nie maksymalizować ich nieprzewidywalność.

PRNG przeznaczony do eksperymentu nie musi być właściwy do haseł, kluczy kryptograficznych ani tokenów bezpieczeństwa. Tam przewidywalność jest wadą, a nie zaletą.

> Tester mówi: „Podaj seed, żebym odtworzył błąd”. Atakujący mówi: „Podaj seed, żebym odtworzył hasło”. To samo słowo, radykalnie inny poziom entuzjazmu zespołu bezpieczeństwa.

Dlaczego w programie używamy modulo?

Surowe wyniki `std::mt19937` są dużymi liczbami bez znaku. W eksperymencie przekształcamy je do zakresu `0–9999`:

```
value = static_cast<int>(generator() % 10'000U);
```

Takie przekształcenie jest proste i odtwarzalne, ale może powodować niewielkie **obciążenie rozkładu** (*modulo bias*), ponieważ liczba możliwych wyników generatora nie musi dzielić się bez reszty przez 10 000. W tej lekcji nie badamy jakości statystycznej losowania, tylko potrzebujemy kontrolowanych danych dla sortowania. Do zastosowań wymagających określonego rozkładu użylibyśmy właściwej dystrybucji i opisali jej implementację.

Seed nie jest całym przepisem

Aby odtworzyć dane, zapisujemy co najmniej:

- nazwę generatora: `std::mt19937`;
- seed;
- liczbę elementów;
- sposób przekształcenia wyniku generatora;
- typ i zakres wartości;
- układ danych: posortowany, odwrócony albo pseudolosowy;
- wersję kodu generatora.

W doświadczeniu pojawi się też **odcisk danych** (*fingerprint*): krótka liczba obliczona na podstawie całego wejścia. Identyczny odcisk jest wygodnym dowodem, że dwa uruchomienia prawdopodobnie użyły tych samych danych.

Odcisk nie zastępuje pełnych danych i nie jest dowodem matematycznym ich równości — różne zbiory mogą teoretycznie mieć ten sam odcisk. Jest natomiast znacznie lepszym dowodem niż zdanie „chyba było to samo”.

🔁 Powtarzalność eksperymentu

Na potrzeby tej lekcji uznamy eksperyment za możliwy do powtórzenia, jeśli inna osoba na podstawie raportu potrafi przygotować to samo wejście, uruchomić tę samą operację w opisanej konfiguracji, uzyskać poprawny wynik i porównać własne obserwacje z naszymi.

Metryczka powinna zawierać:

- nazwę i wersję badanego programu;
- konfigurację Debug albo Release;
- platformę x86 albo x64;
- kompilator i istotne ustawienia;
- rozmiary danych;
- generator, seedy i sposób mapowania wartości;
- liczbę prób;
- kolejność wykonywania przypadków;
- jednostkę i zakres stopera;
- kontrolę poprawności;
- surowe wyniki, a nie tylko jedną wybraną liczbę.

Powtarzalność nie oznacza, że czasy muszą być identyczne. Oczekujemy natomiast identycznych danych, wyników funkcjonalnych i liczników operacji. Czas traktujemy jako obserwację podatną na środowisko.

📚 Trzy układy danych wejściowych

Zbadamy zoptymalizowane sortowanie bąbelkowe dla trzech układów:

🔼 Dane posortowane

```
0 1 2 3 4 5 6 7
```

W pierwszym przebiegu nie następuje żadna zamiana, więc warunek `!swapped` kończy algorytm. Dla `n` elementów wykonujemy około `n` porównań.

🔽 Dane odwrócone

```
7 6 5 4 3 2 1 0
```

Każda para stoi w niekorzystnym porządku. Algorytm wykonuje maksymalną dla tej implementacji liczbę zamian oraz pełną serię przebiegów.

🎰 Dane pseudolosowe

```
4 1 7 1 9 2 6 3
```

Ich dokładny układ zależy od generatora i seeda. Często wymagają wielu przebiegów, ale słowo „pseudolosowe” nie gwarantuje automatycznie jednego, matematycznie określonego przypadku średniego. Dlatego zapisujemy seed, odcisk oraz faktyczne liczniki.

📈 Złożoność — interesuje nas tempo wzrostu

Złożoność obliczeniowa opisuje, jak wraz z rozmiarem wejścia rośnie liczba potrzebnych operacji albo pamięci. Nie mówi wprost, ile mikrosekund program zajmie na konkretnym komputerze.

W zapisie `O(f(n))` litera `n` oznacza rozmiar danych. Notacja dużego O opisuje asymptotyczne ograniczenie wzrostu, czyli zachowanie dla odpowiednio dużych `n`, z pominięciem stałych współczynników i mniej znaczących składników.

Na poziomie tej lekcji będziemy mówić o klasach wzrostu:

Klasa	Intuicja po podwojeniu `n`	Przykład
`O(1)`	liczba operacji nie zależy od `n`	odczyt jednego wskazanego elementu
`O(log n)`	wzrost jest niewielki	wyszukiwanie binarne w posortowanych danych
`O(n)`	około dwa razy więcej pracy	przejście przez wszystkie elementy
`O(n log n)`	nieco więcej niż dwa razy	typowe efektywne sortowania porównaniowe
`O(n²)`	około cztery razy więcej pracy	sortowanie bąbelkowe w niekorzystnych przypadkach

„Około” jest tu ważne. Dla małych danych składniki pomijane w modelu mogą być widoczne, a czas zależy również od implementacji i środowiska.

🔢 Liczba porównań w sortowaniu bąbelkowym

Dla odwróconych danych pętla wykonuje:

```
(n - 1) + (n - 2) + ... + 2 + 1 = n × (n - 1) / 2
```

Dla `n = 500` otrzymujemy:

```
500 × 499 / 2 = 124750
```

Po podwojeniu rozmiaru do `1000` liczba wynosi:

```
1000 × 999 / 2 = 499500
```

Stosunek `499500 / 124750` jest bardzo bliski `4`. To znacznie czystszy dowód kwadratowego wzrostu niż pojedynczy czas zmierzony stoperem.

Dla danych już posortowanych warunek wcześniejszego zakończenia daje `n - 1` porównań i jeden przebieg. Mówimy wtedy o liniowym najlepszym przypadku. Dla danych odwróconych oraz typowych dużych danych pseudolosowych obserwujemy wzrost kwadratowy.

> Stoper mówi, jak długo trwał konkretny bieg. Licznik porównań mówi, ile płotków rzeczywiście ustawiliśmy na torze.

🧠 Złożoność pamięciowa — czego nie wolno ukryć?

Sortowanie bąbelkowe działa **w miejscu**: poza wejściową tablicą potrzebuje tylko kilku zmiennych, dlatego jego **pamięć pomocnicza** ma złożoność `O(1)`.

Cały program nadal przechowuje `n` elementów wejścia, więc łączna pamięć obejmująca dane ma `O(n)`. Zdanie „Bubble Sort potrzebuje tylko `O(1)` pamięci” jest poprawne wyłącznie wtedy, gdy jasno mówimy o dodatkowej pamięci algorytmu.

🧪 Projekt eksperymentu

Program zbada:

- rozmiary `500`, `1000`, `2000` i `4000`;
- dwa seedy: `20260401` i `20260402`;
- dane posortowane, odwrócone oraz pseudolosowe;
- pięć prób czasowych każdego przypadku;
- czas, liczbę porównań, zamian i przebiegów;
- odcisk wejścia i poprawność wyniku względem `std::sort`.

Przygotowanie danych, odcisk oraz wynik wzorcowy znajdują się poza mierzonym fragmentem. W kolejnych próbach zmienia się kolejność trzech układów danych, aby żaden z nich nie był zawsze pierwszy.

Dla jednego rozmiaru, seeda i układu oczekujemy:

- takiego samego odcisku w każdej próbie;
- takich samych liczników porównań, zamian i przebiegów;
- `correct=true`;
- czasów, które mogą się różnić.

💻 Kompletny program bazowy

Program jest przeznaczony dla Visual Studio 2022 i standardu C++17 lub nowszego.

```
#include <algorithm>
#include <array>
#include <chrono>
#include <cstdint>
#include <iostream>
#include <numeric>
#include <random>
#include <string_view>
#include <vector>

constexpr std::array<std::size_t, 4> sizes = { 500, 1000, 2000, 4000 };
constexpr std::array<std::uint32_t, 2> seeds = { 20260401, 20260402 };
constexpr int trial_count = 5;

enum class InputOrder
{
    Sorted,
    Reversed,
    Pseudorandom
};

constexpr std::array<InputOrder, 3> input_orders =
{
    InputOrder::Sorted,
    InputOrder::Reversed,
    InputOrder::Pseudorandom
};

struct SortStats
{
    std::uint64_t comparisons = 0;
    std::uint64_t swaps = 0;
    std::uint64_t passes = 0;
};

struct TrialResult
```

```
{
    std::size_t size;
    std::uint32_t seed;
    InputOrder order;
    int trial;
    long long microseconds;
    SortStats stats;
    std::uint64_t fingerprint;
    bool correct;
};

std::string_view order_name(InputOrder order)
{
    switch (order)
    {
    case InputOrder::Sorted:
        return "sorted";
    case InputOrder::Reversed:
        return "reversed";
    case InputOrder::Pseudorandom:
        return "pseudorandom";
    }

    return "unknown";
}

std::vector<int> make_data(
    std::size_t size,
    InputOrder order,
    std::uint32_t seed)
{
    std::vector<int> values(size);

    if (order == InputOrder::Sorted)
    {
        std::iota(values.begin(), values.end(), 0);
    }
    else if (order == InputOrder::Reversed)
    {
        std::iota(values.begin(), values.end(), 0);
        std::reverse(values.begin(), values.end());
    }
    else
    {
        // Nowy generator uruchomiony z tym samym ziarnem odtwarza ten sam ciąg.
        // Dzięki temu eksperyment można powtórzyć i sprawdzić na tych samych danych.
        std::mt19937 generator(seed);

        for (int& value : values)
        {
            // Operator modulo upraszcza wartości do potrzeb eksperymentu. Nie badamy tutaj
            // jakości rozkładu losowego, więc świadomie pomijamy jego niewielkie odchylenia.
            value = static_cast<int>(generator() % 10'000U);
        }
    }

    return values;
}

std::uint64_t fingerprint(const std::vector<int>& values)
{
    // Odcisk pozwala szybko zauważyć przypadkową zmianę danych wejściowych.
    // Nie służy do bezpieczeństwa i nie jest kryptograficzną funkcją skrótu.
```

```
    std::uint64_t result = 14695981039346656037ULL;

    for (const int value : values)
    {
        // Dla liczby bez znaku przepełnienie jest zdefiniowane modulo 2^64: nadmiarowe
        // bity są odrzucane. Dzięki temu obliczenie pozostaje powtarzalne.
        result ^= static_cast<std::uint32_t>(value);
        result *= 1099511628211ULL;
    }

    return result;
}

SortStats bubble_sort(std::vector<int>& values)
{
    SortStats stats;

    // Pustego wektora nie sortujemy. Sprawdzamy rozmiar przed size - 1, ponieważ
    // std::size_t nie ma wartości ujemnych, a 0 - 1 dałoby ogromną liczbę.
    if (values.size() < 2)
    {
        return stats;
    }

    for (std::size_t end = values.size() - 1; end > 0; --end)
    {
        bool swapped = false;
        ++stats.passes;

        for (std::size_t i = 0; i < end; ++i)
        {
            ++stats.comparisons;

            if (values[i] > values[i + 1])
            {
                std::swap(values[i], values[i + 1]);
                ++stats.swaps;
                swapped = true;
            }
        }

        if (!swapped)
        {
            // Brak zamiany oznacza, że dane są już uporządkowane. Wczesne wyjście pokazuje
            // najlepszy przypadek sortowania bąbelkowego zamiast wykonywać zbędne przebiegi.
            break;
        }
    }

    return stats;
}

TrialResult run_trial(
    std::size_t size,
    std::uint32_t seed,
    InputOrder order,
    int trial)
{
    const std::vector<int> source = make_data(size, order, seed);
    // Odcisk i wynik wzorcowy służą walidacji, a nie są częścią badanego algorytmu.
    // Wykonujemy je poza pomiarem, aby nie zawyżały czasu sortowania bąbelkowego.
    const std::uint64_t source_fingerprint = fingerprint(source);
    std::vector<int> expected = source;
```

```
    std::vector<int> result = source;
    std::sort(expected.begin(), expected.end());

    const auto start = std::chrono::steady_clock::now();
    const SortStats stats = bubble_sort(result);
    const auto stop = std::chrono::steady_clock::now();

    const long long elapsed =
        std::chrono::duration_cast<std::chrono::microseconds>(stop - start).count();

    return TrialResult
    {
        size,
        seed,
        order,
        trial,
        elapsed,
        stats,
        source_fingerprint,
        result == expected
    };
}

int main()
{
    // Metadane zapisują rodzaj danych, rozmiar i ziarno, czyli przepis potrzebny
    // do odtworzenia dokładnie tego samego wejścia pseudolosowego.
    std::cout
        << "engine=std::mt19937\n"
        << "value_mapping=engine_output_modulo_10000\n"
        << "size;seed;order;trial;time_us;comparisons;swaps;passes;fingerprint;correct\n";

    bool all_correct = true;

    for (const std::uint32_t seed : seeds)
    {
        for (const std::size_t size : sizes)
        {
            for (int trial = 1; trial <= trial_count; ++trial)
            {
                for (std::size_t position = 0; position < input_orders.size(); ++position)
                {
                    // Rotujemy kolejność, aby jeden rodzaj danych nie był zawsze mierzony na
                    // nierozgrzanym programie albo po wszystkich pozostałych próbach.
                    const InputOrder order = input_orders[
                        (position + static_cast<std::size_t>(trial - 1)) %
                        input_orders.size()];

                    const TrialResult result = run_trial(size, seed, order, trial);
                    all_correct = all_correct && result.correct;

                    std::cout
                        << result.size << ';'
                        << result.seed << ';'
                        << order_name(result.order) << ';'
                        << result.trial << ';'
                        << result.microseconds << ';'
                        << result.stats.comparisons << ';'
                        << result.stats.swaps << ';'
                        << result.stats.passes << ';'
                        << result.fingerprint << ';'
                        << std::boolalpha << result.correct << '\n';
                }
```

```
            }
        }
    }

    std::cout << "all_correct=" << std::boolalpha << all_correct << '\n';
    return all_correct ? 0 : 1;
}
```

🧭 Jak czytać program?

`make_data` tworzy jeden z trzech układów. Dla danych pseudolosowych za każdym razem konstruuje nowy `std::mt19937` z przekazanym seedem. Dzięki temu generator rozpoczyna od tego samego stanu, zamiast kontynuować ciąg pozostawiony przez wcześniejszą próbę.

`fingerprint` przechodzi po całym wejściu i aktualizuje 64-bitowy odcisk. Arytmetyka liczb bez znaku ma w C++ zdefiniowane zawijanie modulo, dlatego obliczenie jest powtarzalne. Funkcja nie jest używana do zabezpieczeń kryptograficznych.

`bubble_sort` zwraca `SortStats`. Licznik `comparisons` rośnie przed każdym porównaniem sąsiadów, `swaps` przy rzeczywistej zamianie, a `passes` przy rozpoczęciu kolejnego przebiegu zewnętrznej pętli.

`run_trial` przygotowuje źródło, odcisk, kopię do sortowania i wynik wzorcowy przed uruchomieniem stopera. Mierzony fragment obejmuje wyłącznie instrumentowane sortowanie bąbelkowe. Liczniki są częścią badanej implementacji, dlatego nie porównujemy jej czasu bezpośrednio z nieinstrumentowaną funkcją z innej lekcji.

Pętla w `main` rotuje kolejność układów pomiędzy próbami. Dla danych posortowanych i odwróconych seed nie zmienia wejścia, więc ich odciski dla tego samego rozmiaru powinny być identyczne także między seedami. Dla danych pseudolosowych zmiana seeda powinna zmienić odcisk.

🎾 Jak analizować wynik?

Najpierw filtrujemy dane według rozmiaru, seeda i układu. Dla pięciu prób jednego przypadku sprawdzamy:

- czy każdy wiersz ma `correct=true`;
- czy odciski są identyczne;
- czy liczby porównań, zamian i przebiegów są identyczne;
- jak różnią się czasy;
- czy po podwojeniu `n` liczba operacji rośnie około dwa czy około cztery razy.

Przykładowe dane liczbowe w objaśnieniu wzoru nie są Waszymi wynikami pomiaru. Do raportu wpisujecie wartości rzeczywiście otrzymane przez program.

📊 Współczynnik wzrostu

Dla kolejnych rozmiarów obliczcie:

```
współczynnik wzrostu = liczba operacji dla 2n / liczba operacji dla n
```

Jeśli liczba porównań rośnie liniowo, współczynnik będzie zbliżony do `2`. Jeśli kwadratowo — do `4`. Nie obliczajcie współczynnika dla liczby zamian, gdy wartość dla mniejszego rozmiaru wynosi `0`, ponieważ oznaczałoby to dzielenie przez zero.

🧪 Ćwiczenia i eksperyment

Wykonujcie poziomy kolejno. Pomiary czasu przeprowadzajcie w Release bez debuggera. Debugger wykorzystujcie do obserwowania stanu generatora, seeda i liczników na małych danych.

🟢 Poziom A — odtwarzam dane i sprawdzam eksperyment

- Uruchomcie program bazowy i zachowajcie pełny surowy output.
- Dla `n=500`, jednego seeda i każdego układu pokażcie pięć odcisków oraz liczników operacji.
- Wykażcie, które wartości pozostają identyczne między próbami, a które mogą się zmieniać.
- Zmieńcie jeden seed, uruchomcie program ponownie i pokażcie zmianę odcisku danych pseudolosowych.
- Sprawdźcie, czy zmiana seeda wpływa na dane posortowane i odwrócone, oraz wyjaśnijcie wynik na podstawie kodu.
- Zmieńcie na chwilę jedną instrukcję generatora, pokażcie zmianę odcisku, a następnie przywróćcie program bazowy.
- Wyjaśnijcie różnicę między odtworzeniem danych, powtórzeniem wyniku funkcjonalnego i uzyskaniem identycznego czasu.

📦 *Do oddania — poziom A*

Przekażcie poniższy zestaw plików. W miejscu Kryptonim wpiszcie Wasz przydzielony kryptonim TiDPI.

```
Kryptonim\
└── TiDPI_04\
    ├── raport.md
    ├── main.cpp
    ├── wyniki\
    │   ├── A_bazowy.txt
    │   ├── A_zmieniony_seed.txt
    │   └── A_zmieniony_generator.txt
    └── images\  (jeśli używacie obrazów)
```

Zachowajcie pełny output trzech wariantów i przywrócony kod bazowy. Raport ma wskazywać generator, seedy, dokładną chwilową zmianę instrukcji oraz pięć odcisków i liczników dla każdego układu przy n=500. Opiszcie, co jest odtwarzalne, a co może się zmieniać. Oddajecie jedno sprawozdanie z oznaczeniem faktycznie wykonanego zakresu. Nazwy dodatkowych plików pokazane w drzewie są wzorcem organizacyjnym; jeżeli Wasz program używa innych nazw, zachowajcie działające odwołania i wskażcie te pliki w raporcie. Zapis <...> oznacza nazwę do zastąpienia, a nie dosłowną nazwę pliku. Wszystkie obrazy użyte w raporcie umieśćcie w images\ i odwołujcie się do nich względnie, np. ![Dowód D01](images/D01.png). Jeżeli nie używacie obrazów, katalog images\ nie jest wymagany. Pozostałe załączniki również linkujcie względnie.

🛡️ Poziom B — badam tempo wzrostu

- Dla seeda `20260401` opracujcie wyniki wszystkich czterech rozmiarów i trzech układów.
- Zachowajcie surowe czasy pięciu prób, ale jako główny dowód złożoności wykorzystajcie liczbę porównań i zamian.
- Obliczcie współczynniki wzrostu porównań przy każdym podwojeniu `n`.
- Porównajcie uzyskane liczniki dla danych posortowanych z `n - 1`.
- Porównajcie liczniki dla danych odwróconych z `n × (n - 1) / 2`.
- Przygotujcie wykres liczby porównań względem `n` z osobną serią dla każdego układu danych.
- Sformułujcie wniosek o najlepszym i niekorzystnym przypadku, oddzielając teorię od obserwacji czasów.

Ukończenie wyłącznie poziomów A i B pozwala uzyskać najwyżej ocenę 4.

📦 *Do oddania — poziom B*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_04\
    ├── raport.md  (uzupełniony o A–B)
    ├── wyniki\
    │   └── B_surowe.txt
    └── images\
        └── B_porownania.png
```

Dołączcie wyniki czterech rozmiarów i trzech układów dla seeda 20260401, obejmujące pięć prób, czasy, porównania i zamiany. Jeśli pełny output poziomu A już je zawiera, wskażcie ten plik zamiast tworzyć kopię. Współczynniki wzrostu i porównania z podanymi wzorami umieśćcie w raporcie; wykres liczby porównań podlinkujcie względnie.

🔧 Poziom C — sprawdzam granice uogólnienia

Wykonajcie dwa z trzech badań:

- **Wiele seedów.** Dodajcie co najmniej trzy nowe seedy i sprawdźcie, jak zmieniają się porównania, zamiany i przebiegi danych pseudolosowych. Nie nazywajcie jednego seeda „przypadkiem średnim”.
- **Wcześniejsze zakończenie.** Przygotujcie drugą wersję Bubble Sortu bez warunku `!swapped`. Porównajcie liczniki obu wersji dla trzech układów, zachowując identyczne wejścia i kontrolę poprawności.
- **Prawie posortowane dane.** Dodajcie generator danych, w których zamieniono miejscami kilka par. Jawnie opiszcie liczbę i sposób wykonania zaburzeń oraz zbadajcie wpływ na liczbę przebiegów.

Poprawne ukończenie poziomów A–C otwiera drogę do oceny 5.

📦 *Do oddania — poziom C*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_04\
    ├── raport.md  (uzupełniony o A–C)
    └── rozszerzenia\
        └── C\
            ├── <badanie_1>\  (źródła i pełny output)
            └── <badanie_2>\  (źródła i pełny output)
```

Zachowajcie dwa wybrane badania wraz z parametrami: co najmniej trzema nowymi seedami, wariantem bez !swapped albo regułą zaburzania danych prawie posortowanych. Dołączcie liczniki, odciski i kontrolę poprawności odpowiednie do wybranego badania. Opis zmian i interpretację umieśćcie w raporcie.

🚀 Poziom D — własne badanie ponadprogramowe

Zaprojektujcie jedno badanie wykraczające poza rdzeń lekcji. Możliwe kierunki:

- policzenie porównań wykonywanych przez `std::sort` za pomocą własnego komparatora i ostrożne zestawienie tempa wzrostu;
- zapis deterministycznego zestawu do pliku oraz odtworzenie go bez generatora;
- porównanie odcisków i danych na innym kompilatorze lub systemie wraz z analizą zgodności;
- wykres w skali logarytmicznej i oszacowanie nachylenia dla liczby operacji;

- inne pytanie uzgodnione z prowadzącym.

Ocena 6 wymaga samodzielnej hipotezy, kontrolowanych zmiennych, poprawnej implementacji, surowych dowodów, krytycznej interpretacji i obrony pracy. Samo dodanie większego `n` nie jest wiedzą ponadprogramową — jest głównie skutecznym sposobem ogrzewania pracowni.

📦 *Do oddania — poziom D*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_04\
    ├── raport.md  (uzupełniony o A–D)
    └── rozszerzenia\
        └── D\
            ├── <źródła badania>
            └── <dane i surowe wyniki badania>
```

Dołączcie artefakty wynikające z Waszego wyboru: przy zapisie i odtwarzaniu danych także rzeczywisty plik danych; przy innym kompilatorze wyniki i opis obu środowisk; przy analizie nachylenia dane użyte do obliczeń. Wykresy umieśćcie w images\, a hipotezę, metodę, interpretację i ograniczenia w raporcie.

📝 Sprawozdanie

Oddajecie jedno indywidualne sprawozdanie w Markdownie zgodne z zasadami lekcji 01. Raport może zostać przekazany internetowemu agentowi AI do wstępnej analizy, dlatego musi być całkowicie zanonimizowany.

🔒 Obowiązkowa anonimizacja

- użyjcie wyłącznie przydzielonego kryptonimu TiDPI;
- nie umieszczajcie imienia, nazwiska, adresu e-mail ani nazwy konta w treści, nazwie pliku, metadanych, ścieżkach, zrzutach i innych dowodach;
- sprawdźcie właściwości dokumentów DOCX lub PDF i kadrowanie zrzutów;
- mapę `kryptonim TiDPI ↔ uczeń` przechowuje wyłącznie nauczyciel.

🧱 Wymagana struktura raportu

- `Kryptonim TiDPI i wykonany poziom;`
- `Cel oraz hipotezy;`
- `Metryczka środowiska i programu;`
- `Generator, seedy i sposób przygotowania danych;`
- `Kontrola wejścia i odciski danych;`
- `Kontrola poprawności sortowania;`
- `Procedura;`
- `Surowe wyniki;`
- `Liczba operacji i współczynniki wzrostu;`
- `Wykresy;`
- `Interpretacja i ograniczenia;`
- `Wnioski;`
- `Wykorzystanie AI i sposób weryfikacji.`

Każdy dowód oznaczcie identyfikatorem `D01`, `D02` i tak dalej. Tabele muszą zawierać jednostki w nagłówkach, a wykresy — tytuł, opis osi i legendę. Agent analizujący raport powinien móc wskazać konkretny dowód, lecz końcowa interpretacja i ocena należą do nauczyciela.

Użycie AI jest dozwolone. Wskażcie zgodnie z prawdą narzędzie, zakres pomocy oraz sposób sprawdzenia kodu, obliczeń i wniosków. Musicie potrafić wyjaśnić każdy oddawany fragment.

📊 Progi i kryteria oceny

Wyższy próg obejmuje wymagania wszystkich niższych progów. Tabela wskazuje maksymalną możliwą ocenę za poprawnie wykonany, udokumentowany i terminowo oddany zakres.

Maksymalna ocena	Wymagany zakres	Minimalne dowody
2	uruchomiony program i częściowe wykonanie poziomu A	output, seed, przynajmniej jeden odcisk i kontrola `correct`
3	poprawnie ukończony poziom A	porównanie pięciu prób trzech układów, zmiana seeda i wyjaśnienie różnicy między danymi a czasem
4	poziomy A i B	`4 rozmiary × 3 układy × 5 prób` dla jednego seeda, surowe dane, liczniki, współczynniki wzrostu, wykres i uzasadniony wniosek
5	poziomy A–C	dwa badania poziomu C, jawne hipotezy, identyczne wejścia porównywanych wariantów, kontrole poprawności i ograniczenia
6	poziomy A–C oraz wartościowe badanie poziomu D	wiedza ponadprogramowa, samodzielna metoda, powtarzalne dowody, krytyczna interpretacja i obrona pracy

Niezależnie od poziomu oceniane są:

- poprawność generatora, sortowania i kontroli wyniku;
- kompletność przepisu pozwalającego odtworzyć dane;
- rozdzielenie generowania, kontroli i mierzonej operacji;
- zachowanie surowych wyników;
- poprawność obliczeń liczby operacji i współczynników wzrostu;
- rozdzielenie obserwacji, teorii i przypuszczeń;
- anonimizacja, czytelność i oznaczenie dowodów;
- rozumienie oraz umiejętność obrony pracy.

Agent AI wspiera nauczyciela w odnajdywaniu dowodów i porównywaniu raportu z jawnymi kryteriami. Nie wystawia samodzielnie oceny. Materiał przygotowany z pomocą AI, którego nie potraficie zweryfikować albo obronić, nie jest dowodem Waszych umiejętności i może drastycznie obniżyć ocenę, aż do oceny niedostatecznej.

🚧 Typowe pułapki

- zapisanie seeda bez nazwy generatora i sposobu mapowania wartości;
- utworzenie generatora tylko raz i nieświadome kontynuowanie ciągu między przypadkami;
- użycie bieżącego czasu jako seeda bez zapisania jego wartości;
- przekonanie, że ten sam seed zagwarantuje identyczny czas;
- mierzenie generowania, kopiowania lub `std::sort` razem z Bubble Sortem;
- uznanie jednego pseudolosowego zestawu za formalny przypadek średni;
- wnioskowanie o złożoności wyłącznie z jednego pomiaru czasu;
- pomylenie pamięci pomocniczej `O(1)` z całkowitą pamięcią danych `O(n)`;
- dzielenie przez zero podczas obliczania wzrostu liczby zamian;
- przedstawienie przykładowych liczb z lekcji jako własnego pomiaru;
- ujawnienie danych osobowych w raporcie lub dowodach;
- bezrefleksyjne przyjęcie obliczeń albo wniosków agenta AI.

🏁 Co powinniście zapamiętać?

- Generator pseudolosowy jest deterministycznym algorytmem, a seed ustala jego stan początkowy.
- Pełny przepis na dane obejmuje więcej niż seed: generator, mapowanie, rozmiar, układ i wersję kodu.
- Identyczne dane i wynik nie gwarantują identycznego czasu wykonania.
- Odcisk jest użytecznym dowodem zgodności wejścia, ale nie zastępuje danych ani matematycznego porównania.
- Układ danych może radykalnie zmienić pracę tego samego algorytmu.
- Liczba operacji jest czystszym dowodem tempa wzrostu niż pojedynczy pomiar czasu.
- Zoptymalizowany Bubble Sort ma liniowy najlepszy przypadek i kwadratowy przypadek odwrócony.
- `O(1)` opisuje pamięć pomocniczą Bubble Sortu; wraz z wejściem program przechowuje `O(n)` danych.
- Dobry raport pozwala odtworzyć eksperyment, odnaleźć dowody i oddzielić fakt od przypuszczenia.